either block and wait until the rate reduces, or you can simply reject the request as we do in the next listing. The standard HTTP 429 Too Many Requests status code[2] can be used to indicate that rate-limiting has been applied and that the client should try the request again later. You can also send a `Retry-After` header to indicate how many seconds the client should wait before trying again. Set a very low limit of 2 requests per second to make it easy to see it in action. The rate limiter should be the very first filter defined in your main method, as even authentication and audit logging may consume resources.

> **TIP** The rate limit for individual servers should be a fraction of the overall rate limit you want your service to handle. If your service needs to handle a thousand requests per second, and you have 10 servers, then the per-server rate limit should be around 100 request per second.

Open the Main.java file in your editor and add an import for Guava to the top of the file:

```
import com.google.common.util.concurrent.*;
```

Then, in the main method, after initializing the database and constructing the controller objects, add the code in the listing 3.1 to create the RateLimiter object and add a filter to reject any requests once the rate limit has been exceeded. We use the non-blocking `tryAcquire()` method that returns `false` if the request should be rejected.

Listing 3.1 Applying rate-limiting with Guava